CLAUDE CODE

Claude Code 23: W. Edward Deming and The Zero Error Philosophy For Your Workflow

Self-Discovery With /Insights and creating Skills vs Commands vs Claude.MD

SCOTT CUNNINGHAM

FEB 23, 2026

Usually every Monday I update about Codechella Madrid, the third annual workshop on difference-in-differences and panel methods I do in Madrid with CUNEF and some friends But I'm waiting on feedback from something that I wanted to check on before doing anothe official update. But please sign up! And if you're eligible for a student or post-doc discount (and their equivalent!), please do email me at causalinf@mixtape.consulting. In the meantir here's what today's substack is about.

Here's the video walk through of what I did today. It's again about Claude Code. (BTW, the thumb nails are crazy pictures! I don't know how to not do it that way. I should ask Claude Code).

*This is the **23rd installment** in my series on working with Claude Code.* Wow! *The previou posts are collected [here]. If you want to try /insights yourself, it's available in Claude Code today. Thank you for supporting the substack! This is a labor of love, so please consider becoming a paying subscriber. For only a cup of coffee ($5/month), you too can have full ac to everything ranging from difference-in-differences explainers, causal inference discussion Claude Code updates, and all sorts of random stuff over the years. I think there's around 70 800 posts on this thing!*

I Spent This Morning Asking Claude to Analyze Me

This morning, I ran a command called `/insights` and it read through 73 of my Cla Code sessions — six weeks of work — and produced a portrait. Not a report. A portrait. It told me how many messages I'd sent (585), how many lines Claude had

written for me (44,486), what I work on (five domains, from econometrics to course development), and when I work (evenings, mostly, in the margins of my day).

Then it told me what I'm good at and what keeps going wrong. And then it made recommendations of how to improve. But as always, I wanted the information in de format, and as I started working on the deck, I slowly slipped into the process of creating skills versus creating commands, and a process of doing that sort of opene up, which I'll try to explain, but which you can see for yourself in the video walk-through.

Using /`Insights` As a Wellness Checkup / Myers-Briggs Personality Test

Before diving into what happened, let me explain my philosophy. I actually am not i the camp that the goal is to download other people's /`skills`. That's something tha Ethan Mollick said on Linkedin too, which sort of confirmed at least that I am not t only one. I am firmly in the camp that you should be trying to use Claude Code intensively, repeatedly, on actual routine projects like "research" or "teaching", and then use /`insights` (a slash command you type from the Claude Code prompt once you execute it in the terminal — see my video for what I mean if that's new for you). You let Claude Code, in other words, analyze your use, and then generate somethin like a psych profile of how you work, what your comparative advantage is, what you doing great and what you need to work on. And that to me is going to have the high level of success because not everyone needs to be good at everything.

This is the principle of comparative advantage and that's the attitude to take, in my opinion, with the creation of /`skills` and /`commands`. You want to figure out your o style, and then create things for you based on that self-understanding. That's what I mean when I say use /`insights` like it's a Wellness Checkup. Or maybe even more accurately is use it like a Myers-Briggs style personality profile — let it try to figure

out what you're great at and what you're using CC for so that it can maximize there course we want global max, not just local max, but I think this is still the road to th

The Deck Was the Point

But when you use /insights, you get an .html in one of your hidden folders, so I sho the video walk through how to find it by clicking command-shift-period. And while the html is fine, I have basically poisoned my brain to only want decks. It's not so dissimilar to how I only drink IPAs now. I doubt I could do one of those things you people doing at bars where they "drink the menu" by having a beer from the entire menu. All I want to do is drink IPAs. In the same way, I seem to only want decks — good or bad, I use decks to learn more of what I've done, or where I'm going, as dec are sequential flashcards that tell a story. So you'll see in the video walk through no only how I created a self diagnostic report using /insights; I also created a deck usir my particular style of using a command I've made called /compiledeck and a detaile essay on my mixtapetools repository called "rhetoric of decks".

So, Claude did that —it turnd the insights html with all its data into a Beamer presentation — a proper slide deck, following the Rhetoric of Decks philosophy I've been developing, with beautiful TikZ diagrams and custom matplotlib figures and z compile warnings. It had zero compile warnings because my compiledeck comman (before this session, it was only a command — not a skill) insisted it check and rech overfull/overfill/hbox/vbox error messages until there weren't any.

Interestingly, /insights told me that LaTeX presentations were one of my top two w areas (20 sessions out of 73), which didn't surprise me for the reasons I just listed. T decks aren't a side product of my research. They *are* a major part of my research workflow. They are like highlighting a manuscript, or taking notes in a journal, afte reading a paper. They're just so easy to create and so easy to tinker with, and so I us

them religiously, which I suspect means I am now shifting into a new mental mode the world — the deck — without fully realizing it.

What the Portrait Revealed

So here was roughly the diagnostics that /`insights` found. The insights analysis identified a pattern it called "**ambitious delegation with sharp correction**." I give Claude roughly 8 instructions per session. I am pretty sure each time I open a new terminal window, that counts as a new session, but I need to check. Point is, a sessi does not appear to be a project or a project folder. And for me, Claude executes roughly 37 actions per session which it called "a 4.6x multiplier". My personal, subjective style is to set direction and delegate to Claude who does the legwork, an then — this is the important part — **I audit the output aggressively**.

Aggressive auditing appears to be such a strong part of my own personal workflow that Claude flagged it. Twenty-seven times across those 73 sessions, Claude started down the wrong path. Twenty-three times it misunderstood what I asked. Thirty-fo times the code had bugs. I haven't fully automated them away; rather, I have a workflow that inserts myself into "the pipeline" at key points religiously which is h I catch these things. And sometimes the corruption is sharp. The 82% success rate wasn't despite the corrections. It was because of them. And that I think is the featu not the bug — I am inserted into the verification system and that is why my success rate is so high.

The Bézier Problem, or Why Spatial Awareness Matters

But without meaning to, today's video session evolved into what I was wanting to st doing which is use the substack to illustrate, not my skills and commands, but rath how I am going about discovering which ones I need to make for myself so that you

can see for yourself how you might do the same kind of self-reflection with /insight create your own solutions to your own unique problems. There really isn't a one solution in other words — there are universal principles that we all have to follow, f sure. But there are also subjective ones. Not all of us have to take heart medicine to live even though we all have to breathe air and drink water and get enough calories live. A doctor prescribes both, but not everyone needs heart medication. But the healthy living protocol — that is going to follow known biological principles, and given random fluctuations in one's own biology, you'll need to figure out how to tw it until you get to where you need.

So like I was saying — without really meaning to, the narrow issue of creating "the perfect deck" became the test case for me to really shift my workflow and so you'll s that in the video walk through.

With that said, while building the deck to show viewers on the substack what I had found in /`insights`, I kept finding TikZ errors, but they were not the overfull/overf ones as I had already created /compiledeck to double check until those were gone which is part of my "zero error" philosophy, which I encourage you to stop too in yo own workflow. I don't mean zero error in some meta-researcher philosophy though mean iterate until the workflow using AI agents is minimized to zero. Not minimiz — minimized to zero. You must take the position that you will **never tolerate a mistake**. And the only way to never tolerate a mistake is to find them, figure out the causes, and then automate away what you can, and verify verify verify too. It's both/and, not either/or.

The errors I kept finding then were not the typical beamer errors of overfull/overfil but rather they were the less easy to identify (for an LLM) the Tikz errors. Tikz erro do not generate compile errors because they are in coordinate space. They involve, instance, text labels sitting on top of arrows, boxes overlapping boxes, annotations bleeding into neighboring elements. And I kept asking Claude to fix them. And Cla kept missing them. But gradually, we started updating the /`compiledeck` command

figure out the process by which not just this error was fixed, but also the data generating process that created this error was shut down too.

Some were pretty easy to fix, but not all of them. For instance, one label which read "via the terminal" was positioned between two boxes. It, and more like them, surviv **three rounds** of me saying "fix this" before we figured out what was actually wrong The text was wider than the gap between the two boxes. Claude was adjusting the vertical position (moving it up, moving it down) when the problem was horizontal. T text physically didn't fit in the space. So we updated the command markdown to explain a new process of identifying that before it happened.

Tikz Errors and Bézier curves

But then there were the curves which seemed, through repeated trials, to be someth different than the previous thing. These curve errors were labels in space floating o an arrow, and oftentimes it was a curved arrow too. Upon repeatedly talking to Clau about it, we found these were called Bézier curves and they were uniquely creating their own kinds of errors that I could see but which Claude could not see because Claude does not have eyeballs. The issue is pretty simple: TikZ lets you draw curved arrows with a command like `bend left=35,` and those curves follow a mathematic path. But Claude wasn't putting two and two together to track the curve; it kept placing labels in the path of these curves — text sitting right where the arrow swee through. I'd point it out, Claude would fix that one instance, and the same error wo appear on another slide.

This was when we started to inch towards updating /`skills` and /`commands`. We did something that turned out to be the most productive part of the entire session. Inst of just fixing each error, I asked: *Claude, why did you miss this?* No judgement — I wa trying to slam him. I was asking Claude to reflect on the causes of his own failures because maybe if he could see the cause of the failure, he could identify the DGP fo

that failure, and we could surgically go to that DGP and make it not just stop for th one graphic, but for all graphics.

And Claude was honest. It said it couldn't intuit where a Bézier curve passes at a gi point. **It was eyeballing** — estimating based on intuition rather than computing. A when I asked it to audit its own work, it re-ran the same flawed intuition and got th same wrong answer.

But eyeballing is unnecessary with Bézier curves as those follow equations. The fix was a formula. The maximum depth of a curved arrow is `(chord / 2) × tan(bend_angle / 2)`. So once we wrote that down, Claude could compute a num and compare it to the label's position. It was in other words arithmetic, not spatial reasoning. And that's an important point because LLMs are notoriously bad at spat reasoning, so to help it overcome that constraint, you need fixes that are designed f that problem.

And that led us to start creating a taxonomy of such spatial reasoning problems. Fo instance, we found another category: **arrows crossing arrows**. Same underlying issu — Claude couldn't see that a curved return arrow would intersect a vertical branch. was a different formula based on the same principle involving converting the spatia problem into a computational one.

But then we found still a **third category**: an annotation rectangle whose left edge extended into the neighboring box. This one was subtler. No formula would catch i automatically. You just had to notice that `x=3.6` (the rectangle edge) was less than `x=3.8` (the box edge), meaning they overlapped by 0.2cm. Claude had the numbers. just didn't spontaneously compute the spatial implication.

I've read that LLMs struggle with spatial reasoning — the classic example is chess, where the model knows the rules but can't reliably track where pieces are after fifte moves. This is the same thing. Claude knows TikZ syntax perfectly. It just can't hol mental map of where everything is on the slide.

W. Edward Deming and Zero Error Philosophy For Your Workflow

Here is the general lesson that I want you to understand, which I tried to make apparent in the video walk through. Don't see mistakes and failures as bad things. Rather, use them to make lemonade out of lemons. What I mean is, let these mistak guide you and Claude to diagnosing his own reasons for failing, the causes of his ov failures, and the let that discovery lead you to **creating your own skills**.

There's a man named W. Edwards Deming — a statistician who went to Japan after war and taught them something that American manufacturers had been ignoring. T core idea wasn't complicated: **every error is information**. Don't just fix the defect. F out why the defect happened, and change the process so it can't happen again.

That's what we did with the Bézier problem. We didn't just move the label once. Rather we wrote a formula, put it in a reference document, and restructured the ent TikZ verification workflow so that curved arrows get checked first — before anythi else — using arithmetic instead of eyeballing. Then we found the arrow-crossing-arrow problem and added that. Then the annotation-overlap problem. Each failure became a new rule.

By the end of the session, the `tikz_rules.md` file had nine rules and a five-pass verification workflow, organized not by type of error but by order of operations: Bé curves first (because they're the most dangerous and the most systematic), then gap calculations, then label positioning, then everything else, then open the PDF and lo

The file is a collection of what I'd call prosthetic spatial reasoning. Each rule compensates for a specific blind spot in the AI's ability to reason about where thing are on a page. And it'll keep growing. Every time I find a new category of error, we'l add a rule.

Skills vs. Commands, or Why the Container Matters

So, if we adopt this Deming-like philosophy of "zero errors", it means not just to believe it — we have to entrench it. It has to be entrenched inside of the workflow itself, which is both those parts that can be codified into /`skills` and /`commands`, a those parts which must be part of the human verification process. There will **never** a time when there is no human verification because you and I are 100% responsible everything we do as scientists. But we can minimize those errors as much as possib through properly designed workflows such that when do insert ourselves, our time more efficiently used.

But as my errors are produced by a "Scott Cunningham fixed effect", the solutions v need to be designed with me, and not someone else, in mind too. And that's where /`insights` comes in. You can use /`insights` as a Myers-Briggs type of tool that figu you out, and thus helps you figure out solutions that work for you.

One of the things /`insights` revealed was that my compiledeck tool — the set of instructions that tells Claude how to build a slide deck — had been a *command* and *a skill*. The distinction matters as skills and commands cascade through your Claud Code interactions differently.

A command is a single file of instructions that Claude reads once. Think of it as a memo you hand to a research assistant. They read it and do their best. But the prob is that "**do your best" isn't good enough when you need zero errors**. A memo that s "check for TikZ collisions" doesn't work if the RA doesn't know how to measure a collision. The instruction is aspirational, not operational.

So over the process of that video walk through, Claude and I converted it to a *skill*. A skill is different from a command in that skills are a structured directory with multi files. The main file has the operational workflow. A separate file has the TikZ rules with actual formulas. Another file has color palettes extracted from real decks. Another has domain-specific patterns for different types of presentations.

The difference, in other words, between a command and a skill is the difference between telling someone what to do and training them how to do it. And critically the skill lives in a global directory (`~/.claude/skills/`) instead of buried in one project folder, so it's available no matter where I'm working.

That was another friction point `/insights` identified — I'd built the tool in one pla and then couldn't access it from another.

The Comparative Advantage Problem

Here's where I part ways slightly with the "starter pack" philosophy that's popular i the AI productivity world right now. Several have suggested sharing prompt librari and workflow templates — downloading someone else's system and plugging it in. understand the appeal. But I'm suspicious of it, for economic reasons.

For one, I believe in comparative advantage principles which are utterly unique to a person's own production function, which is extremely personal. The insights analys showed me that **my edge is "ambitious delegation with sharp correction"** — I delegate more than most users, but I also audit more aggressively. That's not a transferable template nor should it necessarily be — at least not in the same way. It more of a disposition than a template. Someone who delegates without auditing ge different set of errors requiring their own solution to those errors because the data generating process is different and unique to that person. Someone who audits without delegating never gets to the ambitious projects unless they figure out how solve that problem which is unique to their own style.

The TikZ rules we built today are specific to *my* workflow. I make a lot of slide deck use a lot of TikZ diagrams. I'm particular about visual quality. Someone who prima writes Python scripts and never touches LaTeX would need completely different ru

The /`insights` data would tell them completely different things about where their friction lives.

This is why I think the right model isn't downloading someone else's workflow. It's regularly running /`insights` on your *own* usage, finding *your own* friction points, ar iteratively building *your own* set of skills and rules. The universal principles exist — zero tolerance for errors, convert spatial problems to computational ones, always as why an error happened instead of just fixing it. But the specific implementation is yours.

What I'm Going to Do Now

I'm going to keep using /`insights` regularly — maybe every few weeks — to check on my own patterns. Each time, I expect to find new friction points that I didn't not before, because the old ones will have been fixed. **This is Deming's insight applied individual knowledge work: the process of improvement is itself a process that improves.**

The decks will keep being my thinking tool. The skills will keep growing as I find n blind spots. And I'll keep writing about it here — not because my workflow is the ri one for you, but because the *process* of discovering your workflow is universal even when the result is personal.

Comments Restacks

Write a comment...